TS ACADEMY · MODULE 7

Pandas for Data Analysis

Master the most important Python library for data analysis — from DataFrames to GroupBy operations.

AGENDA

What We'll Cover

- Introduction to Pandas — Series & DataFrames
- Reading Data (CSV, Excel, JSON)
- Exploring DataFrames
- Selecting Data — `loc` and `iloc`
- Filtering & Boolean Indexing
- Handling Missing Data
- GroupBy Operations
- Merging & Joining DataFrames

SECTION 1

Introduction to Pandas

The foundation of the data science workflow in Python.

OVERVIEW

What Is Pandas?

The most important Python library for data analysis. It provides powerful, flexible data structures for working with structured data.

DataFrame
2D labeled data (like a spreadsheet)

Series
1D labeled array (like a column)

I/O Tools
Read CSV, Excel, JSON & more

Analytics
Group, aggregate, pivot data

DATA STRUCTURES

Series — A Single Column

A one-dimensional labeled array. Think of it as a single column with an index.

From a List

```
import pandas as pd sales = pd.Series([100, 150, 200, 175]) print(sales) # 0 100 # 1 150 # 2 200 # 3 175
```

From a Dictionary

```
pop = pd.Series({ 'California': 39538223, 'Texas': 29145505, 'Florida': 21538187 }) print(pop['Texas']) # 29145505
```

DATA STRUCTURES

DataFrame — A Full Table

A 2D labeled structure with columns of different types. Like a spreadsheet or SQL table.

```python
data = { 'name': ['Alice', 'Bob', 'Charlie', 'Diana'], 'age': [25, 30, 35, 28], 'city': ['New York', 'LA', 'Chicago', 'Houston'], 'salary': [70000, 80000, 90000, 75000] } df = pd.DataFrame(data)
```

`df.shape`	`df.columns`	`df.dtypes`
(rows, columns)	Column names	Data types per column

SECTION 2

Reading Data

Load data from CSV, Excel, JSON and more into DataFrames.

READING DATA

Loading CSV Files

```python
# Basic CSV read df = pd.read_csv('sales_data.csv') # With options df = pd.read_csv( 'sales_data.csv', usecols=['date', 'product', 'total_amount'], parse_dates=['date'] )
```

PYTHON

usecols	parse_dates	na_values
Select specific columns	Auto-parse date columns	Custom missing markers

READING DATA

JSON & Other Formats

JSON

```
# Direct read df = pd.read_json('products.json') # Nested JSON import json with
open('data.json') as f: data = json.load(f) df = pd.DataFrame(data['items'])
```

Excel

```
# Requires openpyxl df = pd.read_excel( 'report.xlsx', sheet_name='Sales' ) # Multiple
sheets sheets = pd.read_excel( 'report.xlsx', sheet_name=None )
```

SECTION 3

Exploring DataFrames

Understand your data before you analyze it.

EXPLORING

First Look at Your Data

```python
df.head() # First 5 rows df.tail(3) # Last 3 rows df.info() # Column types, non-null counts df.describe() # Statistics: mean, std, min, max
```

PYTHON

1

`value_counts()`

Count unique values in a column — great for categorical data

2

`isnull().sum()`

Count missing values per column — essential for data quality

SECTION 4

Selecting Data

`loc` & `iloc`

Two ways to access exactly the data you need.

SELECTION

`loc` vs `iloc`

LOC — LABEL-BASED

```
# Single row by label df.loc['E002'] # Row + column df.loc['E002', 'salary'] # Slice (inclusive!) df.loc['E001':'E003', 'name':'dept']
```

ILOC — POSITION-BASED

```
# First row df.iloc[0] # Last row df.iloc[-1] # Rows 1-3, cols 0-2 df.iloc[1:4, 0:3]
```

Key Difference `loc` uses labels (inclusive slicing) • `iloc` uses positions (exclusive end)

SECTION 5

Filtering & Boolean Indexing

Find exactly the rows you need with conditions.

FILTERING

Boolean Indexing

```python
# Single condition electronics = sales[sales['category'] == 'Electronics'] # Multiple conditions: & (AND), | (OR) — use parentheses! filtered = sales[ (sales['category'] == 'Electronics') & (sales['total_amount'] > 200) ] # Using isin() for multiple values regions = ['North', 'Central'] filtered = sales[sales['region'].isin(regions)]
```

Tip Use `~` to negate: `~df['col'].isin(values)` excludes those values

FILTERING

Query & String Methods

`query()` — Cleaner Syntax

```
# Same as boolean indexing result = sales.query( 'category == "Electronics"' ' and
total_amount > 200' ) # With variables min_amt = 300 result = sales.query( 'total_amount
> @min_amt' )
```

`.str` — String Filtering

```
# Contains mouse = sales[ sales['product'] .str.contains('Mouse') ] # Starts with w_prods
= sales[ sales['product'] .str.startswith('W') ]
```

SECTION 6

Handling Missing Data

Real-world data is messy. Pandas makes cleaning it easy.

MISSING DATA

Drop or Fill?

DROP ROWS

```
# Drop ANY missing clean = df.dropna() # Drop only if specific cols missing clean =
df.dropna( subset=['sales_rep', 'price'] )
```

FILL VALUES

```
# Fill with a value df['rating'].fillna(0) # Fill with mean df['price'].fillna(
df['price'].mean() ) # Fill with string df['rep'].fillna('Unknown')
```

Time Series Use `ffill()` (forward) or `bfill()` (backward) to propagate values

SECTION 7

GroupBy Operations

Split → Apply → Combine. The most powerful pattern in Pandas.

GROUPBY

Split → Apply → Combine

Split
Divide data into groups based on a column

1

Apply
Run a function on each group (sum, mean, count)

2

Combine
Merge results back into a new DataFrame

3

PYTHON

```python
# Total sales by region sales.groupby('region')['total_amount'].sum() # Multiple aggregations sales.groupby('region')['total_amount'].agg(['sum', 'mean', 'count'])
```

GROUPBY

Named Aggregations

The recommended approach — clear, readable column names in the result.

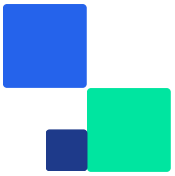
```python
summary = sales.groupby('region').agg( total_sales=('total_amount', 'sum'),
avg_sale=('total_amount', 'mean'), num_transactions=('transaction_id', 'count'),
avg_quantity=('quantity', 'mean') ).round(2)
```

Pro Tip Use `transform()` to keep the original shape: `df.groupby('col')['val'].transform('sum')`

SECTION 8

Merging & Joining

Combine data from multiple sources like SQL JOINs.



MERGING

Four Types of Merge

HOW	KEEPS	SQL EQUIVALENT
`inner`	Only matching rows from both	INNER JOIN
`left`	All rows from left + matches	LEFT JOIN
`right`	All rows from right + matches	RIGHT JOIN
`outer`	All rows from both sides	FULL OUTER JOIN

```python
merged = pd.merge(orders, customers, on='customer_id', how='left')
```

MERGING

Concatenating DataFrames

Stack Rows (Vertical)

```
# Stack two DataFrames stacked = pd.concat( [df1, df2], ignore_index=True ) # df1: rows 0-1 # df2: rows 2-3 # stacked: rows 0-3
```

Add Columns (Horizontal)

```
# Side by side combined = pd.concat( [df1, df3], axis=1 ) # df1: cols A, B # df3: col C # combined: cols A, B, C
```

merge() for joining on keys **concat()** for stacking or appending

SUMMARY

Key Takeaways

DataFrames
Core 2D structure for all data work

Selection
`loc` for labels, `iloc` for positions

Filtering
Boolean masks, `query()`, string methods

GroupBy
Split-apply-combine for aggregations

ESSENTIAL FUNCTIONS

```
pd.read_csv() df.head() df.info() df.describe() df.groupby() pd.merge()
```

WHAT'S NEXT

Keep Practicing

Practice With

- Complete the Jupyter notebook exercises
- Try pivot tables & cross-tabulation
- Explore apply(), map(), and string methods

Coming Up Next

- Data Visualization with Matplotlib & Seaborn
- Creating charts and graphs from your data

Practice with the notebook & ask questions in the group!